



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Linked Lists – The LinkedList Class



Agenda & Reading

- ▶ **Agenda**
 - ▶ The LinkedList class and operations

- ▶ **Online Coursebook:**
 - ▶ <https://onlinetextbook.auckland.ac.nz/17-linked-lists/the-linkedlist-class/>
 - ▶ <https://onlinetextbook.auckland.ac.nz/17-linked-lists/comparing-implementations/>

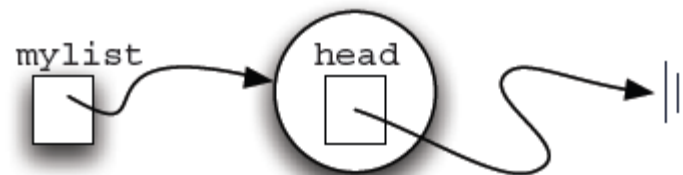


The Linked List

- ▶ A **linked list** is built from a collection of nodes, each linked to the next by explicit references.
- ▶ It must maintain a reference to the first node traditionally referred to as the **head**

- ▶ Examples:

- ▶ An Empty List:



- ▶ A linked list of integers





Operations

- ▶ **List()** creates a new list that is empty.
 - ▶ It needs no parameters and returns an empty list.
- ▶ **add(item)** adds a new item to the list.
 - ▶ It needs the item and returns nothing.
- ▶ **remove(item)** removes the item from the list.
 - ▶ It needs the item and modifies the list. Assume the item is present in the list. The item is not returned.
- ▶ **search(item)** searches for the item in the list.
 - ▶ It needs the item and returns a boolean value.
- ▶ **is_empty()** tests to see whether the list is empty.
 - ▶ It needs no parameters and returns a Boolean value.
- ▶ **size()** returns the number of items in the list.
 - ▶ It needs no parameters and returns an integer.

No checking is
done in the
implementation!



LinkedList Class: Constructor

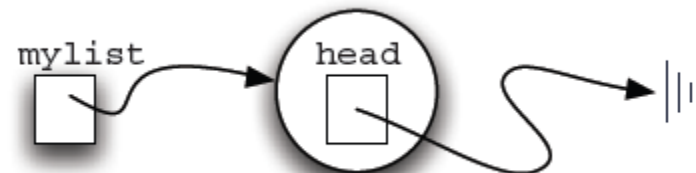
- ▶ The constructor contains
 - ▶ A head reference variable
 - ▶ References the list's first node
 - ▶ Always exists even when the list is empty

```
class LinkedList:  
  
    def __init__(self):  
        self.head = None  
        ...
```

▶ Examples:

- ▶ An Empty List:

```
my_list = LinkedList()
```



- ▶ A linked list of integers

```
my_list = LinkedList()  
for i in range(6):  
    my_list.add(i)
```



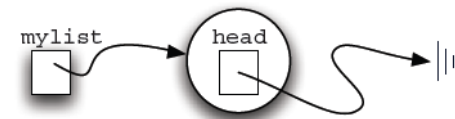


LinkedList Class: is_empty() and size()

► is_empty()

- tests to see whether the list is empty.

```
return self.head == None
```



► size()

- Returns the number of items in the list.
- Traverses the list and counts the number of items

```
current = self.head  
count = 0  
while current != None:  
    count = count + 1  
    current = current.get_next()  
return count
```

count 0->1->2->3->4->5->6





Question

What is the Big-O for the `is_empty()` and `size()` methods of the `LinkedList` class?

- a. $O(1)$, $O(n)$
- b. $O(n)$, $O(1)$
- c. $O(1)$, $O(1)$
- d. $O(n)$, $O(n)$
- e. None of the above.



LinkedList Class: add()

The add() method inserts an item of data at the beginning of a linked list. The item of data is passed as a parameter to the function.

- ▶ Create a new Node and store the new data into it

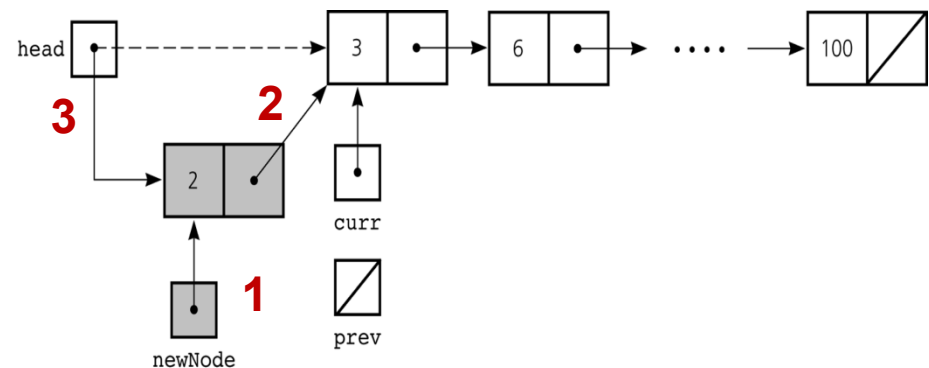
1 `new_node = Node(item)`

- ▶ Connect the new node to the linked list by changing references

- ▶ change the **next** reference of the new node to refer to the **old** first node of the list
- ▶ modify the **head** of the list to refer to the new node

2 `new_node.set_next(self.head)`

3 `self.head = new_node`

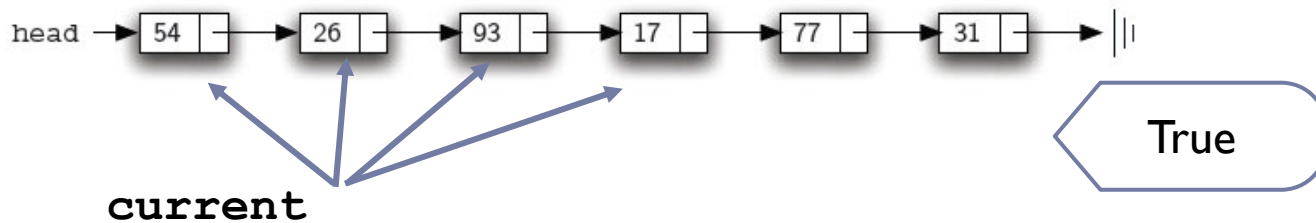




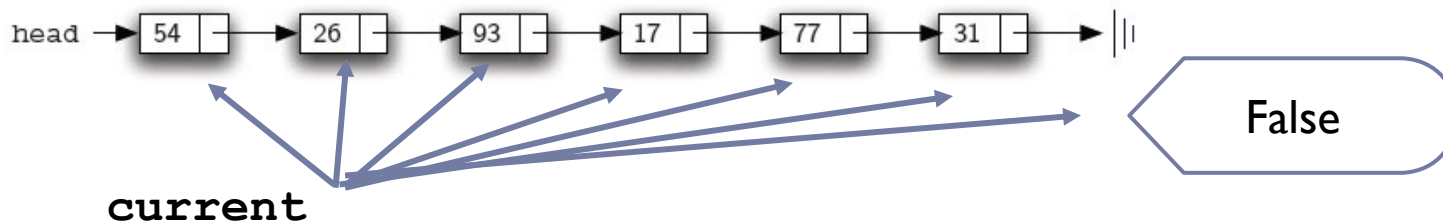
LinkedList Class: search()

The search() method searches for an item in the list. The item is passed as a parameter to the method. The method returns a Boolean.

► Examples: `print (my_list.search(17))`



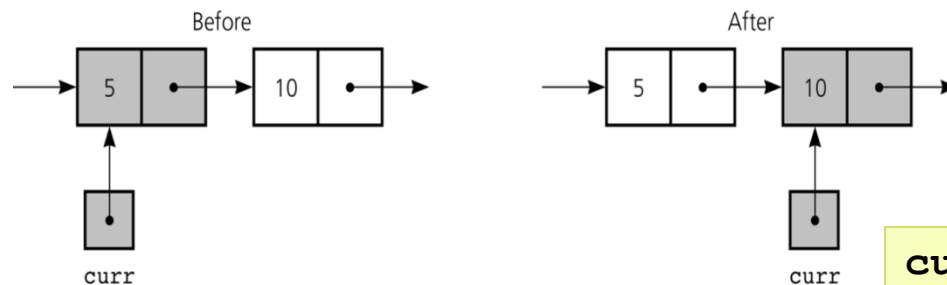
`print (my_list.search(1))`





LinkedList Class: search()

- ▶ To search for an item in a linked list:
 - ▶ set a pointer to be the same address as the **head**,
 - ▶ process the data in the node, (search)
 - ▶ move the pointer to the next node, and so on.
 - ▶ Loop stops when either the item is found or when the next pointer is None.



```
curr = self.head
while curr != None:
    if curr.get_data() == item:
        return True
    else:
        curr = curr.get_next()
return False
```



Question

What is the Big-O for the `add()` and `search()` methods of the `LinkedList` class?

- a. $O(n)$, $O(1)$
- b. $O(n)$, $O(n)$
- c. $O(1)$, $O(1)$
- d. $O(1)$, $O(n)$
- e. None of the above.



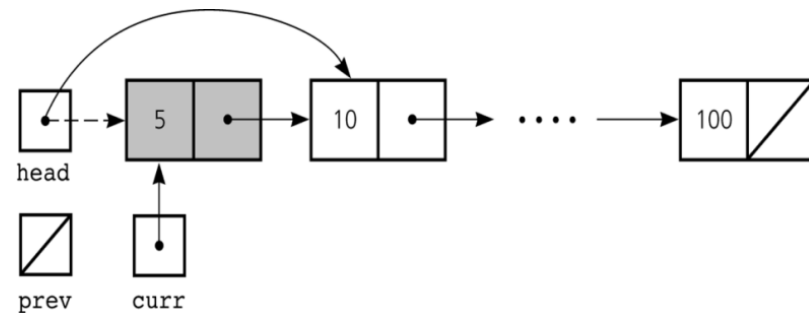
LinkedList Class: remove()

- ▶ Removes the item from the list.
 - ▶ It **needs** the item and modifies the list. Assume the item is present in the list.

- ▶ Examples

- ▶ Delete the first node

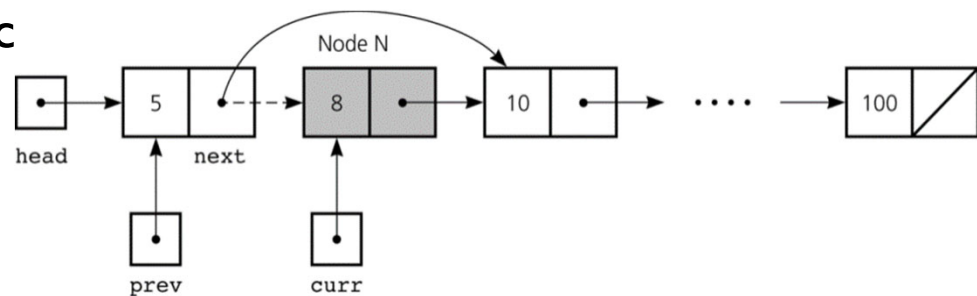
```
my_list.remove(5)
```



- ▶ Delete a node in the middle of the list

- ▶ With **prev** and **curr** referenc

```
my_list.remove(8)
```





LinkedList Class: remove()

- ▶ To delete a node from a linked list
 - ▶ Locate the node that you want to delete (**curr**)
 - ▶ Disconnect this node from the linked list by changing references

- ▶ Two situations:

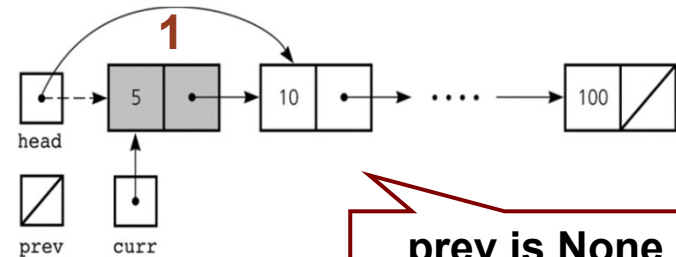
- ▶ To delete the first node
 - ▶ Modify head to refer to the node after the current node

```
self.head = curr.get_next()
```

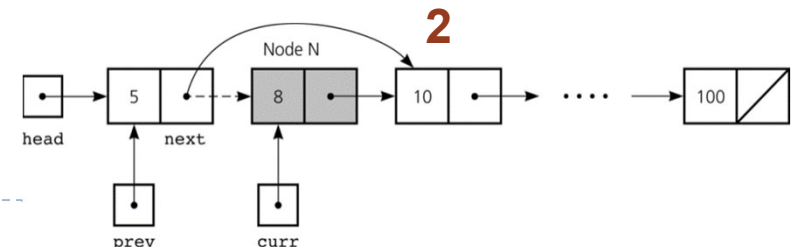
- ▶ To delete a node in the middle of the list

- ▶ Set **next** of the prev node to refer to the node after the current node

```
prev.set_next(curr.get_next())
```



**prev is None
curr references
to the first node**



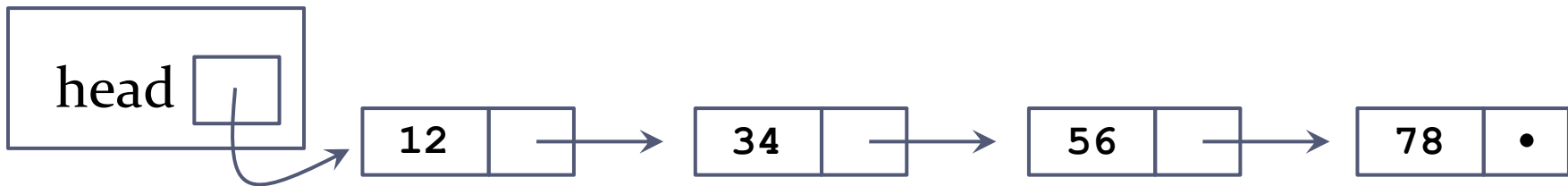


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>



found ☐ previous ☐ current ☐

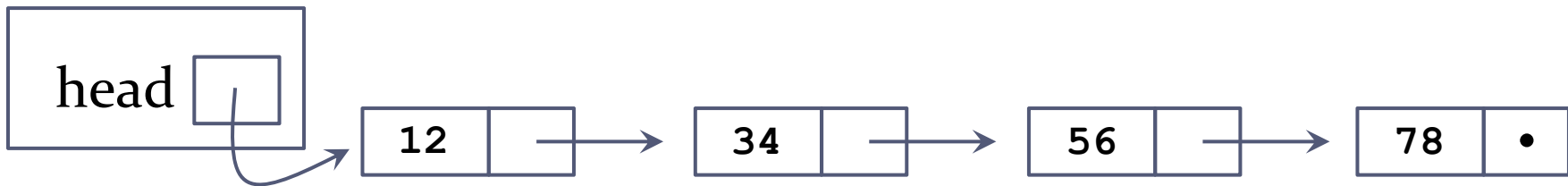


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>



found False previous current

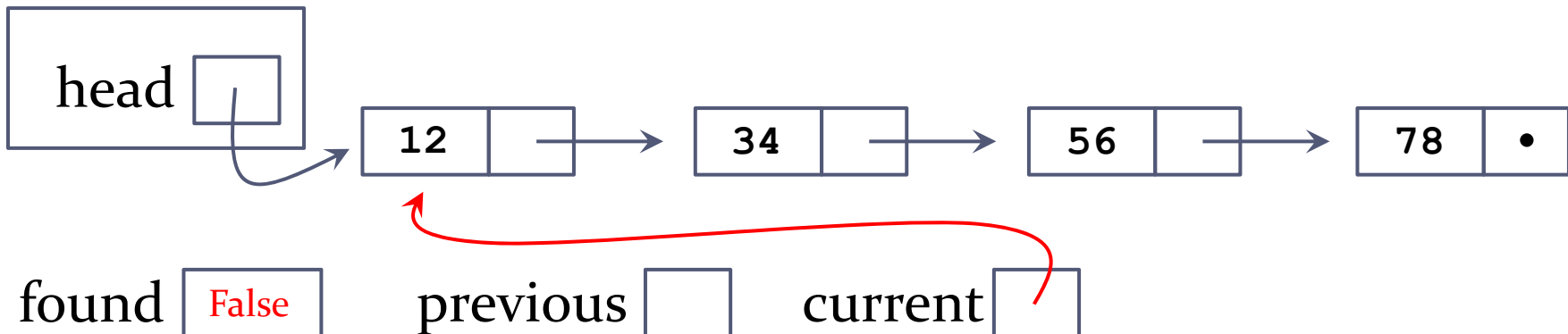


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>



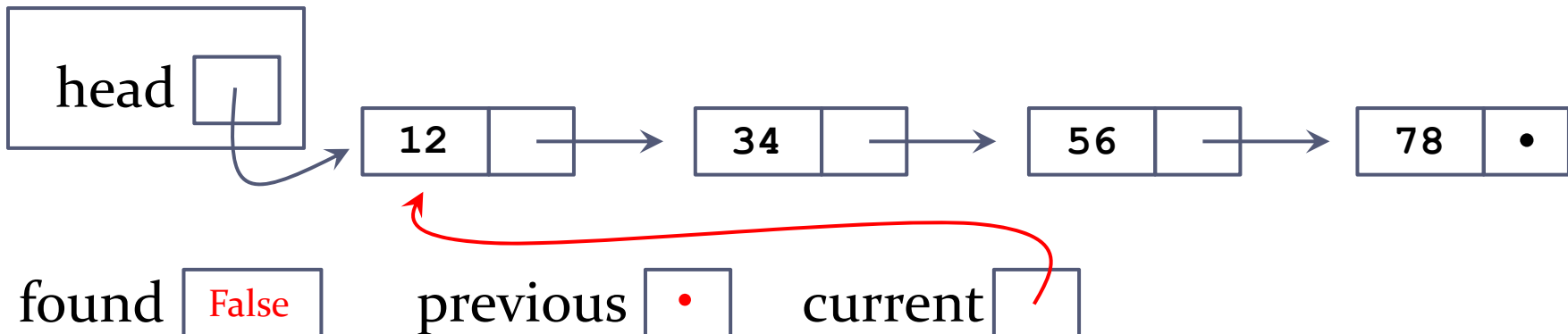


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>



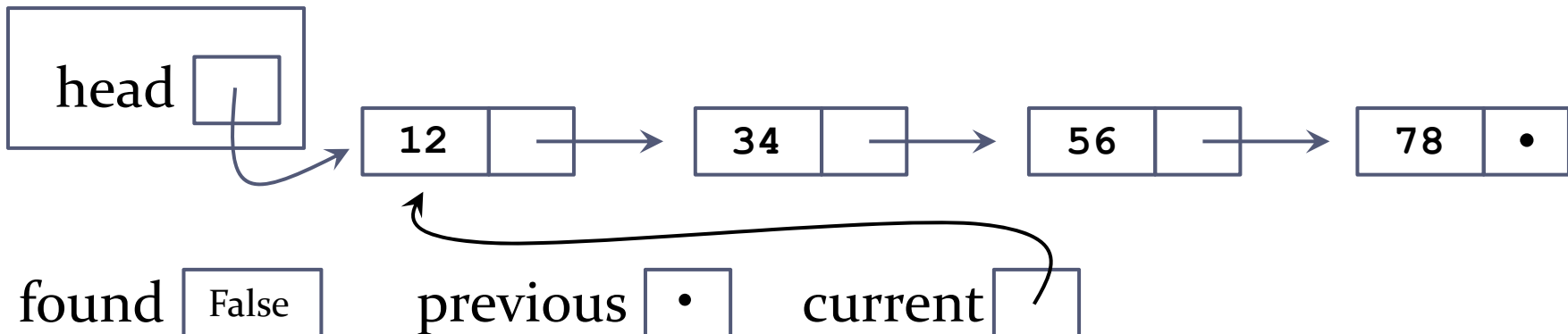


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>



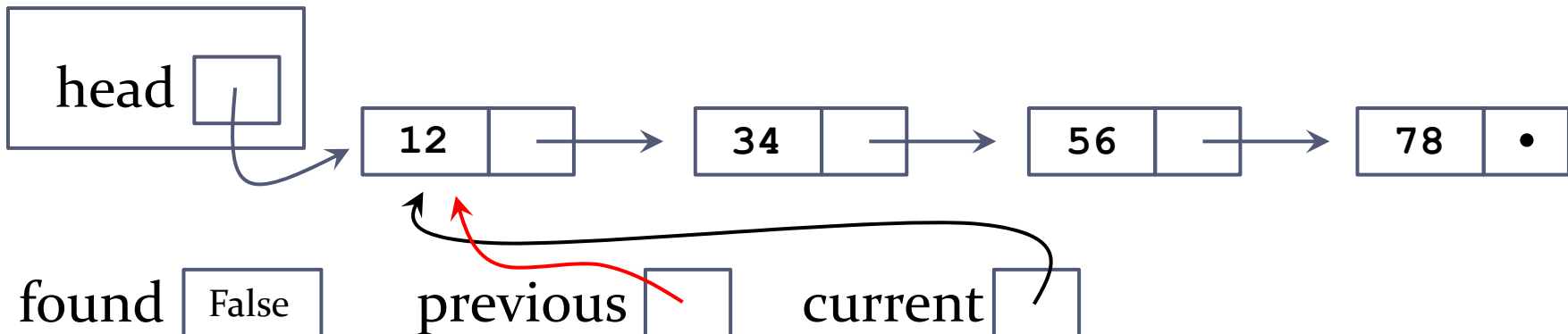


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>



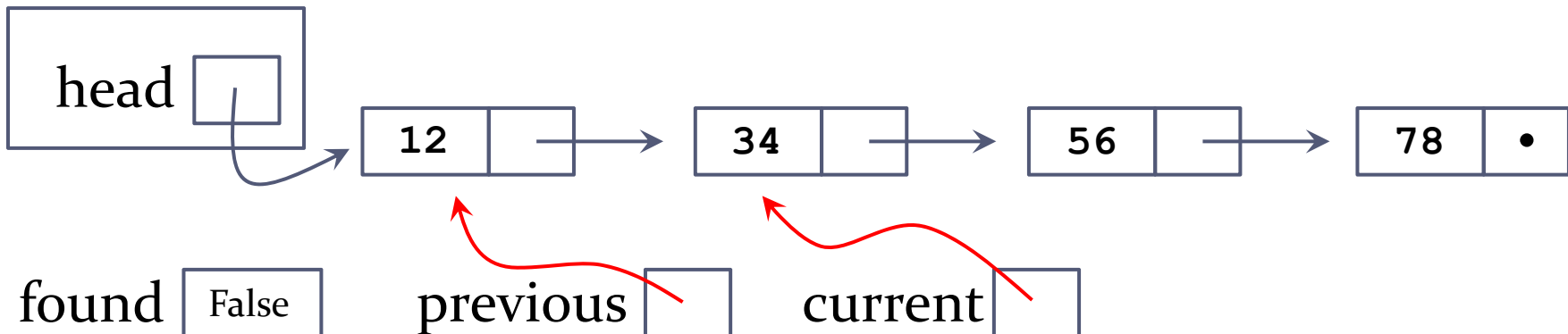


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>



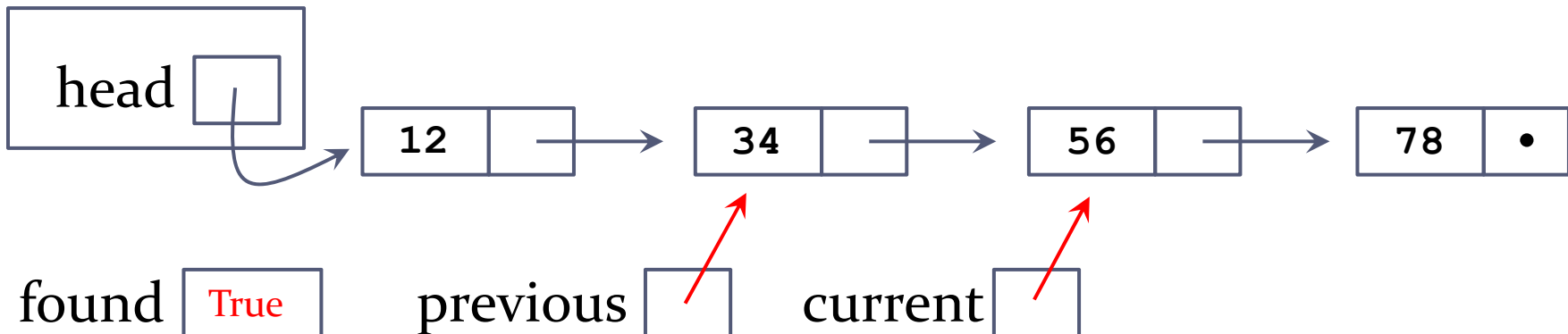


LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        ....  
        ....
```

e.g. remove(56)

<LinkedList>





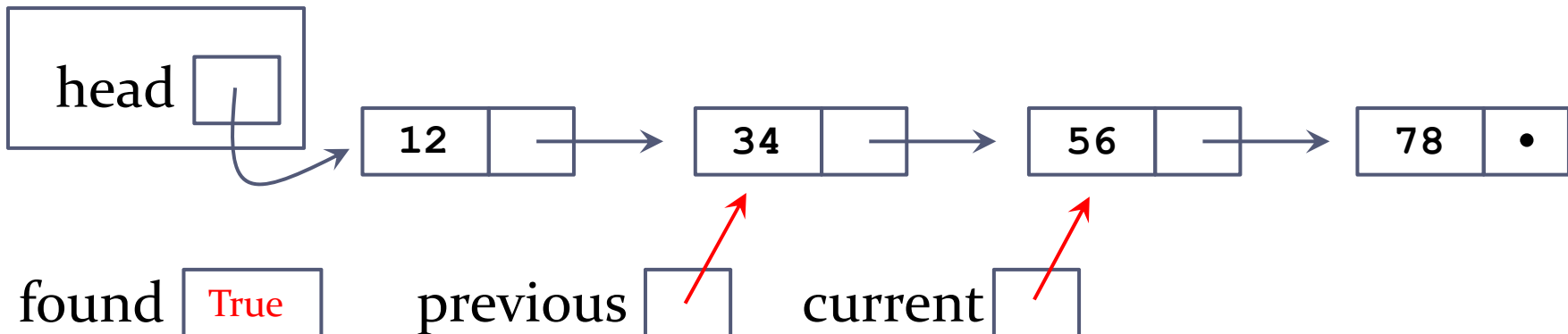
LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        if previous == None:  
            self.head = current.get_next()  
        else:  
            previous.set_next(current.get_next())
```

e.g. remove(56)

This is a node that also has a "previous" node before it. Unlink it from the linked list by setting the next of the previous node to refer to the node after the one being removed

<LinkedList>





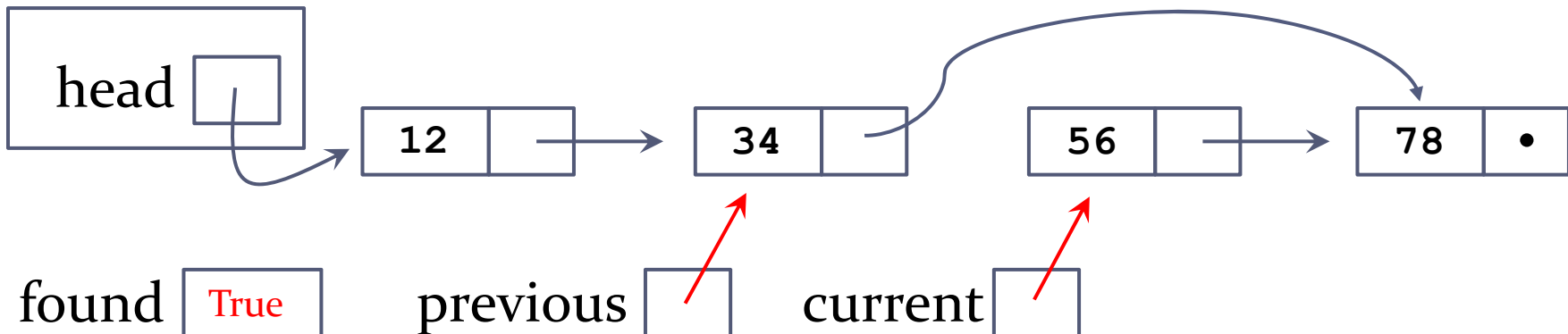
LinkedList Class: remove()

```
def remove(self, item):  
    found = False  
    current = self.head  
    previous = None  
    while current != None and not found:  
        if current.get_data() == item:  
            found = True  
        else:  
            previous = current  
            current = current.get_next()  
    if found:  
        if previous == None:  
            self.head = current.get_next()  
        else:  
            previous.set_next(current.get_next())
```

e.g. remove(56)

This is a node that also has a "previous" node before it. Unlink it from the linked list by setting the next of the previous node to refer to the node after the one being removed

<LinkedList>





Question

- ▶ What is the Big-O for the `remove()` method of the `LinkedList` class?
- a. $O(\log n)$
- b. $O(n \log n)$
- c. $O(n^2)$
- d. $O(n)$
- e. $O(1)$



LinkedList Class: Improvements

- ▶ With a count variable to count the number of items in the list

```
class LinkedListV2:

    def __init__(self):
        self.head = None
        self.count = 0
```

```
def add(self, item):
    new_node = Node(item)
    ...
    self.count += 1
```

```
def size(self):
    return self.count

def is_empty(self):
    return self.count == 0
```

Big-O is $O(1)$

```
def remove(self, item):
    current = self.head
    ...
    self.count -= 1
```



Comparing Implementations

Python List		LinkedList	
if len(my_list) == 0:	O(1)	is_empty()	O(1)
len(my_list)	O(1)	size()	O(1) with <i>count</i> variable O(n) without <i>count</i> variable
append(item)	O(1)	add(item)	O(1) Item added to the beginning of the list
insert (i, item)	O(n)		
remove(item)	O(n)	remove(item)	O(n)
in	O(n)	search(item)	O(n)

Note: Python lists support quite a few additional functions (importantly, functions to access items by index) that a linked list implementation cannot support efficiently.



Exercise 4

- What is the output of the following code?

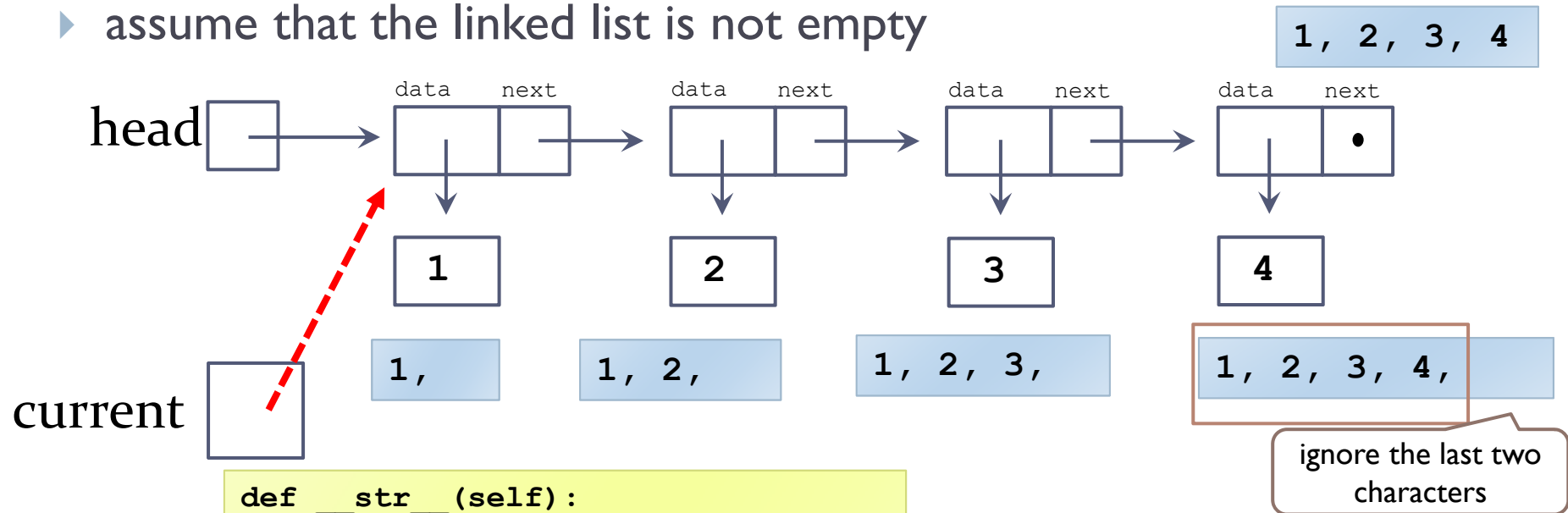
```
my_list = LinkedList()
number_list = [11, 17, 7, 3, 26, 54]
for num in number_list:
    my_list.add(num)
print(my_list.size(), end = " ")
print(my_list.search(17), end = " ")
print(my_list.search(1), end = " ")
my_list.remove(54)
print(my_list.size(), end = " ")
print(my_list.search(54), end = " ")
```





Exercise 5

- ▶ Implement the `__str__(self)` which returns a comma separated string of the values stored in each node
 - ▶ assume that the linked list is not empty



```
def __str__(self):  
    result = ''  
    current = self.head  
    while current:
```

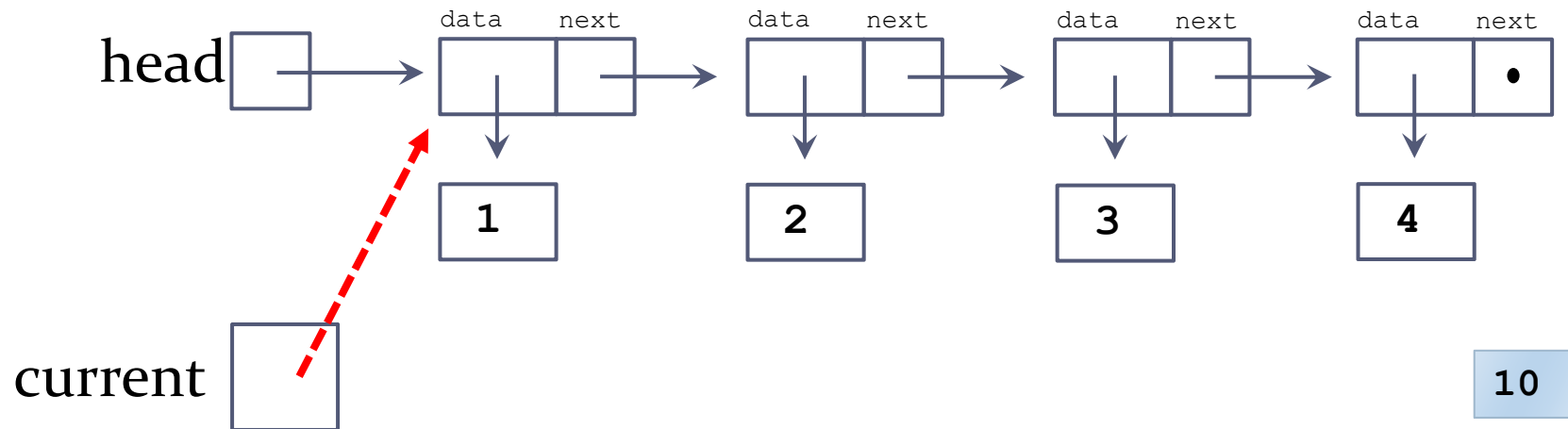
```
        return result[:-2]
```

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.next = None
```



Exercise 6

- Define a method named `get_total(self)` which returns the sum of all the values in a linked list.



```
def get_total(self):  
    total = 0  
    current = self.head  
    while current:  
        total += current.data  
        current = current.next  
    return total
```

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.next = None
```